

Pflichtenheft – Auth Service

Pflichtenheft V1.0

Service-Name:	auth-service
Version:	1.0
Stand:	18.02.2026
Geltungsbereich:	Microservice zur Authentifizierung und Session-Verwaltung innerhalb eines Multi-Mandanten-Portals (je Tenant/Company), inkl. Registrierung/Onboarding (Tenant-Bootstrap), E-Mail-Verifikation, verpflichtender MFA (TOTP), Social Login, JWT Issuer (JWKS), Refresh-Sessions (24h oder 3 Monate), Logout sowie Recovery (Passwort & MFA).

1. Einleitung

1.1 Ziel

Ziel ist die Bereitstellung eines zentralen Auth-Services, der:

- Benutzer-Authentifizierung (Login) für ein Portal ermöglicht
- Registrierung/Onboarding inkl. Tenant-Erstanlage (Bootstrap) abbildet
- E-Mail-Verifikation erzwingt (vor Aktivierung)
- verpflichtende MFA über **TOTP** erzwingt (vor Aktivierung und bei Login)
- Social Login ermöglicht (z. B. Google/Facebook)
- JWT Access Tokens ausstellt (mit `tenant_id` und Multi-Audience `aud`) und Public Keys als JWKS veröffentlicht
- Refresh-Sessions verwaltet (Session-Policy 24h oder 3 Monate)
- Logout (nur aktuelle Session) unterstützt
- Recovery-Prozesse für Passwort und MFA anbietet (Recovery via E-Mail/SMS)

1.2 Abgrenzung / Nicht-Ziele (MVP)

Nicht Bestandteil des MVP:

- Vollständiges Berechtigungsmodell (RBAC/Permissions) → IAM-Service ist Quelle der Wahrheit
- Benutzerprofile/Stammdatenverwaltung → User-Service
- Verwaltung von Unternehmen/Standorten als fachliche Masterdaten → Company-Service
- Template-Verwaltung/Rendering → Template-Service
- Zustellung von E-Mails/SMS/WhatsApp inkl. Retry/Provider-Handling → Messaging-Service/Delivery-Service
- Globale Token-Blacklist in allen Services ("Logout überall sofort") im MVP (stattdessen: Refresh-Revocation + kurze Access Tokens)

1.3 Begriffe

- **Tenant/Company:** Mandant im Portal; Isolation über `tenant_id`.
- **PENDING:** Vorläufiger Zustand während der Registrierung (noch nicht aktiv).
- **ACTIVE:** Aktivierter Zustand; Nutzer darf produktiv arbeiten.
- **Access Token (JWT):** kurzlebiges Token für API-Calls (immer mit `exp`).
- **Refresh Session:** serverseitig verwaltete Session, steuert "24h vs 3 Monate".
- **MFA (TOTP):** Time-based One-Time Password (Authenticator-App).
- **Recovery:** Wiederherstellung bei Verlust von Passwort oder MFA-Device.

2. Rahmenbedingungen / Technologie

2.1 Technologie-Stack (MUSS)

- Java + Spring Boot
- Maven
- MariaDB (MySQL kompatibel)
- REST/JSON
- Konfiguration über `application.properties`
- OpenAPI/Swagger Dokumentation (springdoc)

2.2 Sicherheitstechnologie (MUSS)

- JWT Signaturverfahren: RS256 (oder ES256) mit Key-ID (`kid`)
- JWKS Endpoint für Public Keys
- Passwort-Hashing: BCrypt oder Argon2

2.3 Datenbank-Migrationen (SOLL)

- Schemaänderungen werden versioniert (Flyway empfohlen)
- Hibernate DDL-Auto: `validate` (kein Auto-Create in produktiven Umgebungen)

2.4 OpenAPI/Swagger Dokumentation (MUSS)

Alle Controller und DTOs werden dokumentiert:

- Zweck/Use Case je Endpoint
- DTO-Schemas (Request/Response)
- Beispiele
- Fehlerfälle (Statuscodes + Error DTO)

3. Systemkontext / Integrationen

3.1 Beteiligte Services (MUSS)

- IAM-Service: Autorisierung / Rollen / Permissions (Quelle der Wahrheit)
- User-Service: Benutzerstammdaten/Profil
- Company-Service: Tenant/Company und Standort/Location
- Messaging-Service: Orchestrierung Outbound Messages (Templates, Routing, Retry)
- Template-Service: Rendering von Text/HTML aus TemplateKeys + Variablen
- Delivery-Service: technische Zustellung (E-Mail/SMS/WhatsApp Provider)

3.2 Outbound-Versandkette (MUSS)

Für Verifikations- und Recovery-Kommunikation gilt:

- Auth → **Messaging** → **Template** → **Delivery**

Auth übergibt an Messaging:

- TemplateKey (z. B. `EMAIL_VERIFY`, `PASSWORD_RESET`, `MFA_RECOVERY`)
- Empfänger (E-Mail / Telefonnummer)
- Variablen (z. B. Link, Code, TenantName)
- Korrelation (`correlationId`)

4. Fachliche Anforderungen

4.1 Mandantenmodell / Tenant-Bootstrap (MUSS)

- Benutzer erstellt Tenant aktiv im Registrierungsprozess (Firma/Unternehmen + Standort).
- Der **erste registrierte Benutzer** eines neuen Tenants wird automatisch **Tenant-Admin**.
- Weitere Benutzer dürfen nur durch den Tenant-Admin des jeweiligen Tenants hinzugefügt/eingeladen werden.

4.2 Registrierung / Onboarding (MUSS)

4.2.1 Ablauf (MUSS)

1. Registration Start

- Anlage PENDING-User (Auth + User-Service)
- Anlage PENDING-Tenant/Company und PENDING-Standort/Location (Company-Service) als „Dummy“
- Versand E-Mail-Verifikation (Messaging/Template/Delivery)

2. E-Mail-Verifikation

- Nutzer bestätigt Verifikationslink (Token)

3. MFA Enrollment (TOTP)

- Nutzer richtet TOTP ein (QR/Secret, Code-Bestätigung)

4. Aktivierung

- User wird ACTIVE
- Company/Location werden ACTIVE (Dummy wird „scharf geschaltet“)
- IAM: Tenant-Admin Zuweisung + Standardrollen/Permissions werden final gesetzt

4.2.2 Zeitfenster / Cleanup (MUSS)

- Registrierung muss innerhalb von **24 Stunden** abgeschlossen werden.
- Wird die Registrierung nicht innerhalb von 24h abgeschlossen, werden PENDING-Daten gelöscht:
 - User (inkl. Credentials/Identities, MFA PENDING, Tokens)
 - Tenant/Company (Dummy)
 - Standort/Location (Dummy)

4.3 E-Mail-Verifikation (MUSS)

- Verifikation erfolgt über einen zeitlich begrenzten Token (max. 24h im Registrierungsprozess).
- Token wird serverseitig **nur gehasht** gespeichert.
- Resend ist möglich, aber rate-limitiert.

4.4 MFA (TOTP) (MUSS)

- MFA ist verpflichtend.
- Ohne erfolgreiches MFA Enrollment darf kein User ACTIVE werden.
- Login erfordert nach Primary Auth immer eine MFA-Prüfung (TOTP).

4.4.1 Backup/Recovery (SOLL)

- Backup-Codes SOLL (einmalige Anzeige, nur Hash speichern).

4.5 Login (MUSS)

4.5.1 Login-Methoden (MUSS)

- Passwort-Login (E-Mail + Passwort)
- Social Login (Google/Facebook u. a.)

4.5.2 Login-Flow (MUSS)

1. Primary Auth (Passwort oder Social)
2. MFA Challenge (TOTP)
3. Session anlegen (Refresh Session) entsprechend Session-Policy
4. Access Token ausstellen

4.6 Session-Policy (MUSS)

Beim Login kann der User wählen:

- **24 Stunden** Session
- **3 Monate** Session

Technische Umsetzung:

- Access Token ist **kurzlebig** (immer `exp` gesetzt)
- Refresh Session bestimmt die Gesamtdauer (24h oder 3 Monate)

4.7 Logout (MUSS)

- Logout beendet **nur die aktuelle Session** (Refresh Session).
- Keine globale Abmeldung über alle Geräte im MVP.

4.8 Passwort-Reset (MUSS)

4.8.1 “Passwort vergessen” (MUSS)

- Reset wird über E-Mail (oder optional SMS) gestartet.
- Response ist neutral (“Falls Konto existiert ...”), um Enumeration zu vermeiden.
- Reset Token:
 - kurz gültig (10–30 Min)
 - gehasht gespeichert
 - one-time use

4.8.2 Passwort ändern (eingeloggt) (SOLL)

- Step-Up MFA (TOTP) erforderlich.
- Nach Passwortänderung: Sessions widerrufen.

4.9 MFA Recovery (MUSS)

- Recovery via E-Mail/SMS als “Rescue”, nicht als regulärer 2. Faktor.
- Nach erfolgreicher Recovery:
 - alte MFA Konfiguration invalidieren
 - neues MFA Enrollment erzwingen
 - Sessions widerrufen

5. JWT / Token-Contract (MUSS)

5.1 Access Token – Header (SOLL)

- `alg`: RS256 (oder ES256)
- `kid`: Key-ID (für Rotation)
- `typ`: JWT

5.2 Access Token – Claims (MUSS)

Pflicht:

- `iss` (String)
- `sub` (String, `userId`)
- `aud` (Array, Multi-Audience)
- `iat` (Number)
- `exp` (Number) **immer gesetzt**
- `jti` (String)
- `tenant_id` (String)

Scopes:

- `scp` (Array) und/oder `scope` (String)
Empfehlung: `scp` ist führend, `scope` optional.

5.3 Access Token – Claims (SOLL)

- `sid` (Session-ID)
- `amr` (Array, enthält `"totp"`)
- `auth_time` (Number)
- `subject_type` = `"USER"`

5.4 Audience (aud) – Multi-Audience (MUSS)

`aud` enthält u. a.:

- `iam-service`
- `user-service`
- `template-service`
- `company-service`

5.5 Tenant-Regel (MUSS)

`tenant_id` ist Quelle der Wahrheit. Wenn Request zusätzlich Tenant-Angaben enthält (Header/Path/Query), muss dieser Tenant dem Token-`tenant_id` entsprechen, sonst Zugriff verweigern.

5.6 Service Tokens (SOLL)

Für interne Calls (Auth → Template/Messaging/Delivery) kann Auth kurzlebige Service Tokens ausstellen:

- `subject_type = "SERVICE"`
- `sub = "auth-service"`
- `aud` enthält Zielservice
- `tenant_id` gesetzt (wenn Zielservice Tenant-Kontext erzwingt)
- `exp` kurz (2–5 Minuten)
- `scp` minimal

6. API Spezifikation (REST)

6.1 Public API unter `/api/v1/*` (MUSS)

Registrierung

- `POST /api/v1/registration/start`
- `POST /api/v1/registration/verify-email`
- `POST /api/v1/registration/mfa/totp/enroll`
- `POST /api/v1/registration/mfa/totp/confirm`

Auth / Session

- `POST /api/v1/auth/login`
- `POST /api/v1/auth/mfa/verify`
- `POST /api/v1/auth/refresh`
- `POST /api/v1/auth/logout`

Passwort

- `POST /api/v1/password/forgot`
- `POST /api/v1/password/reset`
- `POST /api/v1/password/change` (SOLL)

MFA Recovery

- `POST /api/v1/mfa/recovery/start`
- `POST /api/v1/mfa/recovery/confirm`

Key Discovery

- GET /.well-known/jwks.json

7. Datenmodell (logisch)

7.1 AuthCredential (MUSS)

- userId
- loginEmail (unique)
- passwordHash
- status (PENDING/ACTIVE/DISABLED/LOCKED)
- failedAttempts, lockedUntil
- createdAt, modifiedAt

7.2 AuthIdentity (MUSS)

- userId
- provider (LOCAL/GOOGLE/FACEBOOK/...)
- providerSubject (unique je provider)
- email (optional)
- createdAt

7.3 MFAConfig (MUSS)

- userId
- totpSecret (verschlüsselt gespeichert)
- enabled
- backupCodesHash (SOLL)
- updatedAt

7.4 Session (Refresh) (MUSS)

- sid
- userId
- tenantId
- refreshTokenHash
- expiresAt (24h oder 3 Monate)
- revokedAt
- lastUsedAt, createdAt

7.5 Tokens (MUSS)

- verificationTokenHash
- passwordResetTokenHash
- mfaRecoveryTokenHash
- jeweils: userId, expiresAt, usedAt, createdAt

8. Sicherheit / Nicht-funktionale Anforderungen

8.1 Passwort-Sicherheit (MUSS)

- Hashing BCrypt/Argon2
- keine Klartext-Secrets in Logs
- Passwort-Policy (SOLL): Mindestlänge etc.

8.2 Rate Limiting / Abuse Schutz (MUSS)

- Login Brute-Force Schutz
- Registrierung, Resend, Password Reset, MFA Recovery jeweils rate-limitiert

8.3 Token-Transport (SOLL)

- Access Token via Bearer Header
- Refresh Token als HttpOnly Secure Cookie (Web)
- CSRF Schutz über SameSite/CSRF Token/Origin Checks

8.4 Observability (SOLL)

- correlationId Logging
- Actuator health/metrics

9. Sonderfälle & Ausnahmefälle (MUSS)

Folgende Sonderfälle sind umzusetzen und zu dokumentieren (Verhalten, Fehlercodes, Logging/Audit, Retry/Compensation):

- Resend Verifikation (Mail nicht angekommen) + Rate Limit + neutrale Antwort
- Registrierung erneut starten nach Ablauf (24h) + vollständiges Cleanup
- Idempotency/Doppelklick/Retry bei kritischen POST-Endpunkten
- Race Conditions zwischen Cleanup und Verifikationsabschluss (atomare Statuswechsel)
- Duplikat E-Mail (ACTIVE User) – keine neue Tenant-Erstellung

- TenantKey Kollisionen
- Teilfehler in verteilten Abläufen (User/Company/IAM/Messaging/Delivery) inkl. Kompensation
- Provider-Sonderfälle (Social Provider ohne verifizierte E-Mail, E-Mail geändert, Provider down)
- Account Linking Regeln (Passwort/Google/Facebook)
- Clock Skew Toleranz (iat/exp)
- Key Rotation/JWKS Caching (mehrere aktive Keys)
- Refresh Token Theft/Reuse (Rotation/Reuse-Detection)
- Statusänderungen während Session (User/Tenant disabled) – spätestens bei Refresh prüfen
- Password Reset als Umgehung für MFA: Reset ändert Passwort, setzt MFA nicht zurück
- MFA Enrollment Abbruch/Fortsetzen bis 24h
- MFA Recovery Missbrauch (Cooldown/Rate Limit/Notifications)
- E-Mail Änderung (falls unterstützt): Re-Verify + Konflikte
- Delivery/Template Fehler (Template missing/render fail, Bounce, Delivery down): Retry/Status

10. Fehlerformat (SOLL)

Standard Error DTO:

- timestamp
- status
- errorCode
- message
- correlationId
- path
- details[]

11. Abnahmekriterien (Auszug)

MUSS:

- Registrierung erstellt PENDING User + PENDING Tenant/Location und triggert Verifikationsmail (Auth→Messaging→Template→Delivery).
- Aktivierung nur nach E-Mail-Verifikation + TOTP Enrollment.
- Cleanup nach 24h löscht PENDING-Daten zuverlässig.

- Login ist immer 2-stufig (Primary + TOTP).
- Session-Policy 24h und 3 Monate funktionieren via Refresh Session.
- Logout beendet nur die aktuelle Session.
- JWT Contract eingehalten (iss/sub/aud/iat/exp/jti/tenant_id + scopes; Multi-aud).
- JWKS liefert Public Keys inkl. kid; Services können validieren.
- Sonderfälle umgesetzt (Idempotency, Teilfehler, Reuse-Detection, Rate Limits).

SOLL:

- Password Change mit Step-Up MFA.
- Backup Codes.
- Monitoring/Actuator.

12. Anhang A – API Beispiele (Request/Response)

A0) Gemeinsame Konventionen

Headers (SOLL):

- X-Correlation-Id
- Idempotency-Key (MUSS bei kritischen POSTs)

Error DTO Beispiel:

```
{
  "timestamp": "2026-02-18T12:34:56Z",
  "status": 400,
  "errorCode": "VALIDATION_ERROR",
  "message": "Field 'email' is invalid",
  "path": "/api/v1/auth/login",
  "correlationId": "c-123",
  "details": [
    { "field": "email", "issue": "must be a well-formed email address" }
  ]
}
```

A1) Registrierung – Beispiele

POST /api/v1/registration/start

Headers:

- Idempotency-Key: <uuid>
- X-Correlation-Id: <string>

Request:

```
{
  "tenant": { "tenantKey": "acme-gmbh", "companyName": "ACME GmbH" },
  "location": { "name": "Hauptstandort", "country": "DE", "city": "Bremen" },
  "user": { "email": "user@example.com", "password": "*****", "displayName":
"Max Mustermann" }
}
```

Response 201:

```
{
  "registrationId": "REG_01J...",
  "userId": "USR_01J...",
  "tenantId": "TEN_01J...",
  "status": "PENDING_EMAIL_VERIFICATION",
  "expiresAt": "2026-02-19T12:34:56Z"
}
```

POST /api/v1/registration/verify-email

Request:

```
{ "token": "verif_abc123" }
```

Response 200:

```
{ "registrationId": "REG_01J...", "status": "PENDING_MFA_ENROLLMENT" }
```

POST /api/v1/registration/mfa/totp/enroll

Request:

```
{ "registrationId": "REG_01J..." }
```

Response 200:

```
{
  "totp": {
    "issuer": "Portal",
    "accountName": "user@example.com",
    "otpauthUri": "otpauth://totp/Portal:user@example.com?
secret=BASE32...&issuer=Portal"
  }
}
```

POST /api/v1/registration/mfa/totp/confirm

Headers:

- Idempotency-Key: <uuid>

Request:

```
{ "registrationId": "REG_01J...", "totpCode": "123456" }
```

Response 200:

```
{
  "status": "ACTIVE",
  "userId": "USR_01J...",
  "tenantId": "TEN_01J...",
  "backupCodes": ["ABCD-EFGH", "IJKL-MNOP", "QRST-UVWX"]
}
```

A2) Login / Session – Beispiele

POST /api/v1/auth/login

Headers:

- Idempotency-Key: <uuid>

Request:

```
{ "email": "user@example.com", "password": "*****" }
```

Response 200:

```
{ "loginTransactionId": "LTX_01J...", "mfaRequired": true }
```

POST /api/v1/auth/mfa/verify

Headers:

- Idempotency-Key: <uuid>

Request:

```
{
  "loginTransactionId": "LTX_01J...",
  "totpCode": "123456",
  "sessionPolicy": "HOURS_24"
}
```

Response 200:

```
{
  "accessToken": "eyJhbGciOiJSUzI1NiIsImtpZCI6Ii4uLiJ9...",
  "tokenType": "Bearer",
  "expiresIn": 900,
  "session": { "sid": "SES_01J...", "policy": "HOURS_24", "expiresAt": "2026-02-19T12:34:56Z" }
}
```

POST /api/v1/auth/refresh

Request:

```
{}
```

Response 200:

```
{ "accessToken": "eyJ...", "tokenType": "Bearer", "expiresIn": 900 }
```

POST /api/v1/auth/logout

Request:

```
{}
```

Response 204: (kein Body)

A3) Passwort – Beispiele

POST /api/v1/password/forgot

Headers:

- Idempotency-Key: <uuid>

Request:

```
{ "email": "user@example.com" }
```

Response 202:

```
{ "status": "IF_ACCOUNT_EXISTS_EMAIL_WILL_BE_SENT" }
```

POST /api/v1/password/reset

Headers:

- Idempotency-Key: <uuid>

Request:

```
{ "token": "reset_abc123", "newPassword": "*****" }
```

Response 200:

```
{ "status": "PASSWORD_CHANGED" }
```

A4) MFA Recovery – Beispiele

POST /api/v1/mfa/recovery/start

Headers:

- Idempotency-Key: <uuid>

Request:

```
{ "email": "user@example.com", "channel": "EMAIL" }
```

Response 202:

```
{ "status": "IF_ACCOUNT_EXISTS_RECOVERY_WILL_BE_SENT" }
```

POST /api/v1/mfa/recovery/confirm

Headers:

- Idempotency-Key: <uuid>

Request:

```
{ "token": "mfa_rec_abcl23" }
```

Response 200:

```
{ "status": "MFA_RESET_REQUIRED", "next": "TOTP_ENROLLMENT_REQUIRED" }
```

A5) JWKS – Beispiel

GET /.well-known/jwks.json

Response 200:

```
{
  "keys": [
    { "kty": "RSA", "kid": "auth-key-2026-01", "use": "sig", "alg": "RS256",
      "n": "...", "e": "AQAB" }
  ]
}
```

13. Anhang B – Idempotency-Regeln (MUSS)

B1) Pflicht-Endpunkte für Idempotency-Key

- /registration/start
- /registration/verify-email
- /registration/mfa/totp/confirm
- /auth/login
- /auth/mfa/verify
- /password/forgot
- /password/reset
- /mfa/recovery/start
- /mfa/recovery/confirm

B2) Regel

Gleicher Key + gleicher Endpoint + gleicher Kontext → gleiche Antwort.

Bei Payload-Abweichung: 409 IDEMPOTENCY_CONFLICT.

14. Anhang C – Kompensation & Retry Matrix

C1) Prinzipien (MUSS)

- Keine verteilte ACID-Transaktion über Services.
- Saga/Compensation + Statusmaschinen + Outbox/Retry.
- Auth orchestriert Registrierungsschritte und steuert Statuswechsel.

C2) Matrix

Schritt	Aktion	Downstream	Fehler-Szenarien	Retry	Kompensation / Korrektur
R1	PENDING User anlegen	User-Service	Timeout/5xx	ja (limited)	falls Company schon angelegt: Company/Location PENDING löschen
R2	PENDING Tenant/Location anlegen	Company-Service	Timeout/5xx	ja (limited)	falls User schon angelegt: User PENDING löschen
R3	Auth-Credentials/Identity anlegen	Auth DB	DB Error	ja (transaktional)	Registrierung abbrechen; kompensierende Deletes in User/Company
R4	Verifikationsmail anstoßen	Messaging	Timeout/5xx	ja (Outbox/Retry)	Registrierung bleibt PENDING; Resend möglich; Ablauf nach 24h Cleanup
R5	Verify-Token prüfen + Statuswechsel	Auth DB	invalid/expired/used	nein	klare Fehlercodes; bei expired: neu registrieren
R6	TOTP Secret erzeugen	Auth DB	DB Error	ja	Status bleibt PENDING_MFA
R7	TOTP Confirm + Aktivierung	User/Company/AM	Teilfehler	ja (kontrolliert)	Status ACTIVATION_IN_PROGRESS; Retry oder Rollback auf PENDING + Cleanup
L1	Primary Login	Auth/Provider	invalid/locked	nein	Lockout/Ratelimit
L2	MFA Verify + Session create	Auth DB	DB/Timeout	ja	Session nur bei Erfolg anlegen
S1	Refresh	Auth DB	revoked/expired	nein	401/403/410; neuer Login
P1	Forgot Password send	Messaging	Timeout/5xx	ja	neutral; erneut möglich (Rate Limit)
P2	Reset Password	Auth DB	invalid/expired	nein	Fehlermeldung; neuer Forgot
M1	MFA Recovery send	Messaging	Timeout/5xx	ja	neutral; erneut möglich (Rate

Schritt	Aktion	Downstream	Fehler-Szenarien	Retry	Kompensation / Korrektur Limit/Cooldown) Fehlermeldung; neuer Start
M2	MFA Recovery confirm	Auth DB	invalid/expired	nein	

C3) Statuszustände (MUSS)

- PENDING_EMAIL_VERIFICATION
- PENDING_MFA_ENROLLMENT
- ACTIVATION_IN_PROGRESS
- ACTIVE
- optional: EXPIRED

C4) Cleanup Race (MUSS)

- Aktivierung ist atomar (DB Transaktion).
- Cleanup löscht nur, wenn Status weiterhin PENDING und `expiresAt < now`.
- Optional: Grace 60s.

C5) Retry/Backoff (SOLL)

- Retry nur bei transienten Fehlern (Timeout/5xx).
- exponentieller Backoff.
- Dead-letter nach Max-Versuchen.

15. Anhang D – Beispiel Access Token Payload

```
{
  "iss": "https://auth.portal.local",
  "sub": "USR_01J...",
  "aud": ["iam-service", "user-service", "template-service", "company-service"],
  "iat": 1766000000,
  "exp": 1766000900,
  "jti": "JWT_01J...",
  "tenant_id": "TEN_01J...",
  "scp": ["user:read", "template:read", "auth:session"],
  "sid": "SES_01J...",
  "amr": ["pwd", "totp"],
  "auth_time": 1766000000,
  "subject_type": "USER"
}
```